

Contents

1. Introduction	1
2. Evolution of the Theoretical Model	2
2.1. Simplification of the Event Log Model	2
2.2. Asynchronous Model of Secondary Inputs	3
3. DBMS Core Implementation	4
3.1. Overall Architecture	4
3.2. Core Data Structure	5
3.3. Data Localization	5
3.4. Object Routing	7
4. Faden Virtual Machine Implementation	7
4.1. General Structure	7
4.2. Compilation	8
4.3. LocValue Value Types	8
4.4. Gear Implementation	8
5. Testing	10
5.1. Target Scenario	10
5.2. StateGraph	10
5.3. Collaborative Text Editing	12
5.4. End-to-End Testing	12
5.4.1. Pure Rust	12
5.4.2. Faden Integration	13
5.4.3. Version Control System	13
5.4.4. Testing Results	14
6. Conclusion	14
Bibliography	14
7. Appendix	15
7.1. Log of Automated Testing Run Results	15

1. Introduction

Within the framework of previous research works, a functional-reactive database architecture was proposed and theoretically justified, using an event log as the single source of truth [1] and utilizing reactive rules (Gear) to materialize a single final state of the system regardless of the order of event processing, ensuring eventual consistency. A theoretical model was formed, its prototype in Haskell was implemented, the rule specification language (Faden) was

designed and implemented, and the specification of the data storage system and efficient parallel query processing was proposed.

The present work is devoted to the practical implementation of the designed architecture in Rust. As part of the work:

1. The core functionality of the DBMS cores has been implemented. A strict core isolation paradigm has been implemented, minimizing contention for access to shared data and ensuring the horizontal scalability of the system.
2. The data transmission and routing protocol has been implemented.
3. The Faden virtual machine has been implemented with integration into the DBMS, allowing the description of transformation rules in a specialized language.
4. End-to-end testing has been conducted: the end-to-end operation of the pipeline has been demonstrated, including the compilation of the Faden source code, launching the DBMS, storing incoming user data in RAM, and processing queries using the example of a version control system for text documents that supports inviting new users to a branch, publishing changes, and merging branches.

2. Evolution of the Theoretical Model

During the practical implementation of the model, theoretically justified in the previous work, a number of decisions were made aimed at simplifying and increasing the practicality of the model. This section lists the main changes.

2.1. Simplification of the Event Log Model

As discussed earlier, updating the internal state of the database by a user is carried out through sending events of an arbitrary format, which are stored in the DBMS log and later processed by reactive rules. The previously described model utilized a reference system in which each event could:

1. Declare several new global objects.
2. Reference objects defined within other events.

A key feature of this method is the natural deduplication of the data store through a clear distinction between the concepts of “data” (objects) and “operations” (events) that operate on these events. However, a detailed analysis revealed a serious flaw in this approach. In the context of a model with strict core isolation, it is assumed that each core has exclusive ownership of the processed data, and the processing of incoming queries is performed predominantly locally, with limited communication with other cores to ensure

the highest performance. In such a system, it is natural to assume that each core, in addition to storing its own partition of events, is forced to completely store all their dependency objects. This significantly limits the potential for data deduplication in multi-core systems while maintaining multiple events referencing the same object. The implementation of this mechanism within the current prototype was deemed impractical due to the high complexity of the software implementation, overhead costs, and limited practical potential.

Alongside this, the previous work considered a client-authoritative model, in which:

- All events are signed by their authors using the BLS12-381 cryptographic signature.
- Clients have full control over the content of the events they create, including the timestamp, which makes it vulnerable to intentional manipulation and falsification.
- The server is perceived as an untrusted party that stores and relays events but is not the source of truth.

Such a model remains promising but is associated with high overhead for asymmetric cryptography and weakened guarantees about the event log, which leads to increased complexity in implementing transformation rules and increased vulnerability of the system to attacks through temporal branch falsification. The current implementation has adopted a server-authoritative model, in which all sent events belong to the destination server, and the temporal branch is perceived as a trusted source of information. The disadvantage of this approach is the increased complexity of synchronizing a single state in the context of a trusted cluster with conflict resolution for the representation of the same event.

2.2. Asynchronous Model of Secondary Inputs

To materialize the final state, the architecture utilizes Gear reactive rules distributed across different cores. Each Gear has access to events on its host core and can access the results of other Gears on the current or other cores.

The previous work proposed a proactive model in which a user request to a Gear triggers a pipeline involving from one to several Gears located on different cores. This algorithm allows returning the latest version of the data but is associated with difficulties:

- Significant increase in latency due to the need for inter-core communication.

- Increased overall utilization of all system cores, leading to performance degradation under high load.
- High implementation complexity with a dynamic dependency graph.

Since the goal of the work is to design a system with eventual consistency, an asynchronous approach is proposed as the primary model, where the processing of a user request is performed by only one core, utilizing locally cached results of other cores and updating the cache in the background after processing high-priority requests.

3. DBMS Core Implementation

This section is devoted to describing the practical implementation of the DBMS core in Rust.

3.1. Overall Architecture

The architecture of the developed DBMS is based on the thread-per-core model with strict memory isolation (shared-nothing), similar to that used in ScyllaDB [2]. Each thread is pinned to a specific core and owns its own set of data: a dedicated event log partition, indexes, and a Gear cache. Communication between cores is carried out exclusively through message-passing channels.

The DBMS is embedded as a library in other Rust projects. The DBMS is launched via the `Db` struct, which spawns several threads, passing them channels for inter-core communication. The operation of the DBMS is tied to the `Db` struct, and the DBMS automatically stops in the object's destructor. Also, upon launch, a `DbHandle` struct is provided, which is used to send user requests into the system, as shown in Figure Figure 1, and read the result.

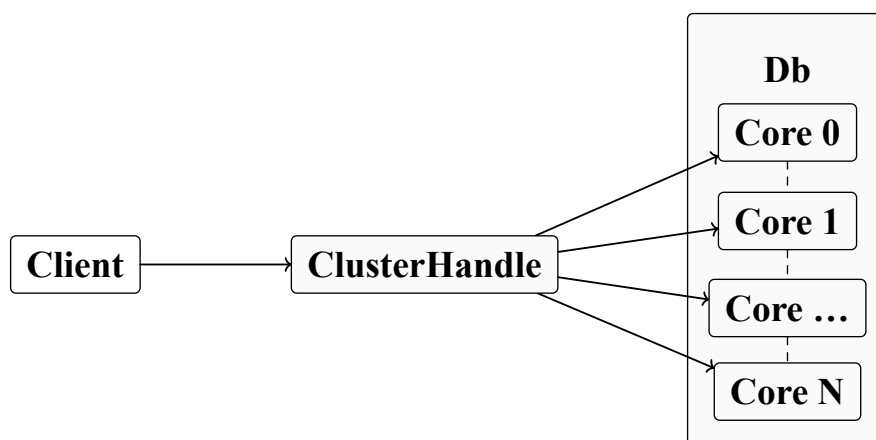


Figure 1: DBMS Architecture with Memory Isolation

Note: the library is implemented to be polymorphic with respect to the Gear execution environment. By default, the Faden execution environment is used to define Gear, allowing dynamic loading of definitions and automatically implementing a portable data storage format.

3.2. Core Data Structure

The Core object is the state repository of a single core. It contains:

- A local context (LocCtx). For each global identifier (user, sender device, event, event group), LocCtx associates a compact local identifier implemented via an 8-byte monotonic counter. LocCtx also directly contains the events stored on the core.
- A single base context FadenModule, identical for all cores.
- Gear instances cache: each Gear is associated with its internal state (“cache”) used to store the internal state between runs. According to the insignificance of the cache requirement discussed in early works, this storage is optional but allows avoiding a complete recalculation of the state with every request.
- Event index by groups.
- Cache of Gear results from other cores.
- Channels for accessing other cores for asynchronous cache updating.

Each core receives messages over two communication channels:

- The **high-priority channel** receives write events (PostEvents), requests to run Gear (RunGear), and a command to shut down (Shutdown).
- The **low-priority channel** is used for asynchronous inter-core synchronization of the secondary input cache. Through this channel come synchronization requests from other cores (SecondaryRequest) and responses in the form of updated data (SecondaryResponse).

Each core in a loop processes all signals of the high-priority channel and, if none are present, starts processing signals of the low-priority channel.

3.3. Data Localization

As discussed earlier, each DBMS core functions in its own LocCtx context, assigning its own identifiers to external data, which may not match. The rejection of strict identifier matching is motivated by the fact that in the considered implementation, data is aggressively partitioned across cores, and a significant part of it does not need to be processed by other cores at all. This approach eliminates the need for inter-core communication to synchronize identifiers and ensures dense data storage.

In situations where there is a need to transfer data that includes references to local objects as part of requests between server cores or over a telecommunications network (for example, as part of communication between a client device and a DBMS device), **localization** is utilized.

Localization is a protocol that allows translating local objects tied to the sender's LocCtx into a dynamically formed portable WireLocCtx context, and then translating the objects back from this context to the local one.

WireLocCtx contains a subset of data from the sender's LocCtx, the minimum necessary for successfully processing the request, and contains complete, globally unique information about:

- users mentioned in the request;
- sender devices mentioned in the request;
- events mentioned in the request.

To construct WireLocCtx objects from the local context and translate requests into it, the WireLocCtxBuilder structure is used, which automatically extracts local identifiers from the request, resolves them in the LocCtx context, supplements the constructed WireLocCtx with the required data, and returns an updated request bound to the constructed context.

To read the data transmitted over the network on the receiver side, the WireLocCtxMerger structure is utilized, which localizes the request into the core context and imports missing data.

A visualization of this protocol is presented in Figure Figure 2.

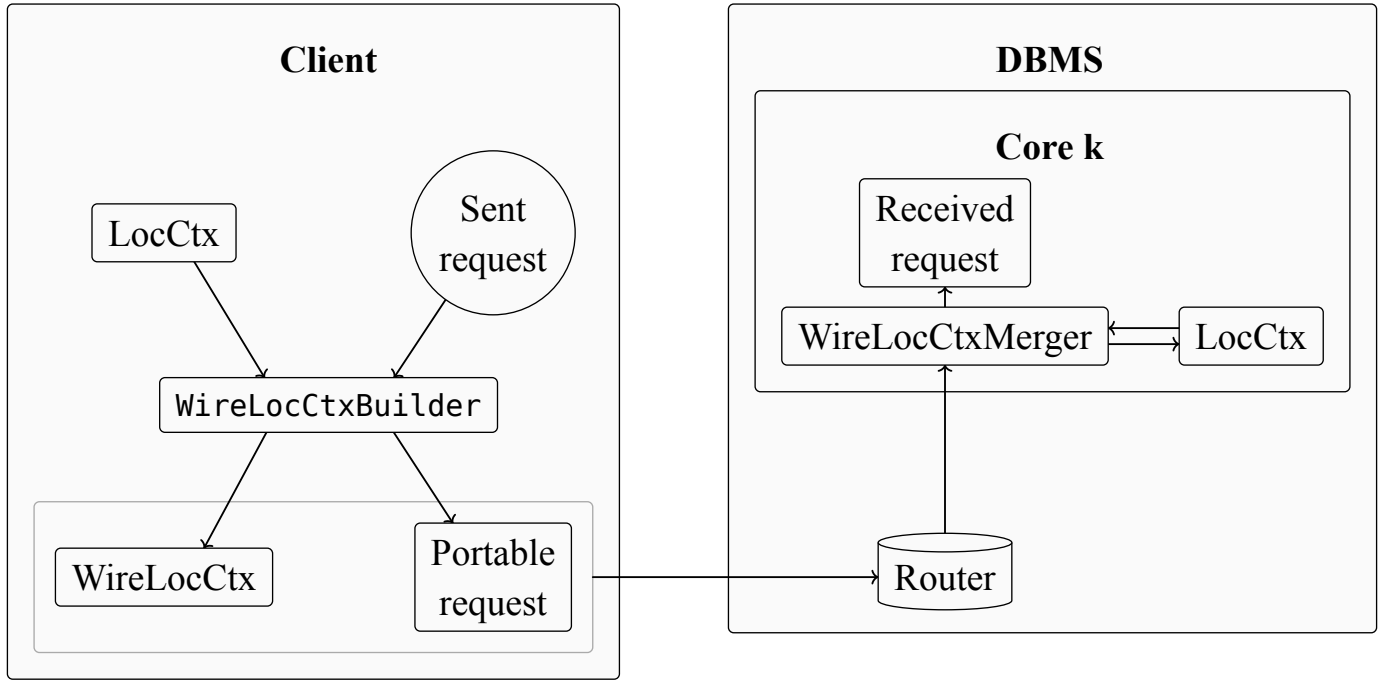


Figure 2: Localization Protocol

The advantage of the protocol lies in the absence of the need for additional requests to synchronize the context. Its disadvantage is the need for double message localization (on the sender's side and on the receiver's side), as well as an increase in packet size.

3.4. Object Routing

As described in the previous work, the routing of events to cores is performed based on group fields, which allows grouping related events on a single core. Each event is assigned a 4-byte `GlobalCoreId` number, calculated as a deterministic blake3 hash of a set of group fields (using globally stable identifiers instead of local ones). The target core is determined by calculating the remainder of the division of `GlobalCoreId` by the total number of DBMS cores.

4. Faden Virtual Machine Implementation

In previous works, the Faden language project was proposed and implemented as a language for describing the logic of transformation rules. This section is devoted to the integration of the Faden virtual machine into the DBMS project.

4.1. General Structure

The integration of the Faden execution environment is represented by the following modules:

- **Compiler:** calls the external binary file `fadeno-lang` to compile source code files.
- **Deserializer:** converts the received code into an internal representation, including an instruction set, constant table, tag table, and module ranges.
- **Virtual machine:** a stack-based virtual machine capable of executing generated code to process data represented as the sum-type `LocValue`.
- **Bridge:** a component that provides the integration of Fadeno into the DBMS as an environment for describing and executing rules with automatic support for the data localization protocol.

4.2. Compilation

To compile Fadeno source files, the previously created Haskell compiler is used, capable of transforming source files into an abstract syntax tree and compiling it into a sequence of instructions. Upon launch, the DBMS starts the compiler as a child subprocess, passing the path to the compiled file containing the description of key event types and Gears, and reads the serialized compiled module through standard output, first ensuring the presence of definitions for the key data types required for the correct functioning of built-in functions.

4.3. LocValue Value Types

To represent the data operated on by Fadeno code, the virtual machine implements the `LocValue` sum-type. This type includes both key Fadeno objects and objects related directly to the DBMS domain, such as local identifiers.

Since `LocValue` can contain local identifiers, a recursive localization protocol is implemented for this type, which completely traverses the entire tree and performs the replacement of mentioned local identifiers.

4.4. Gear Implementation

Gear objects are implemented as a special case of `LocValue`, which can also be transmitted over a telecommunications network or between cores, and are used as ordered/hashable keys when caching internal Gear state between runs.

An example of Fadeno code describing a new event type and the Gear attached to it is presented in Listing Listing 1.

```

// Description of the user "invite" event
/: EventId
Invite = mk_event_type (._
{ .tag = .Invite
// The "group field" of the invite event denotes `Branch`,
// that is, the branch to which the invite is performed.
| .group = Branch
// The message body denotes the user identifier.
// Internally represented by LocUserId.
| .body = UserId
})

// Description of the function that maps each branch to an invites
counter Gear.
/: Fun (Branch) -> Gear Int
invites_count = \branch. mk_gear
{ // Primary input description: sets an event table of type
`Invite`,
// bound to the relevant branch, as the primary input.
// This Gear will be automatically bound to the core where
// all events from this group reside.
.primary = { .tag = .event_table | .type = Invite | .group =
branch }
// Initial cache state description. In this case, it stores
// the `query` object, used to read changes in the group, and
// the internal counter number.
| .initial_cache = { .query = mk_query | .out = 0 }
// Step function, taking as input the previous internal state of
the Gear (its cache)
// and the event table.
| .step = \cache primary _secondary.
// Request added and removed events.
res = query_delta cache.query primary
// Update the counter.
out = (cache.out + (list_length res.delta.added)) - (list_length
res.delta.removed)
// Return the final counter and the updated internal state.
in { .cache = { .query = res.query | .out = out } | .out = out }
}

```

Listing 1: Example of using Faden code to describe an event and its associated Gear

5. Testing

To verify the correctness of the system, two levels of testing were developed: unit tests of components and end-to-end scenario tests that verify the architectural invariants of the complete system.

5.1. Target Scenario

To investigate the applicability of the implementation for solving practical problems, the task of implementing a simple version control system for collaborative document editing is considered. The target scenario consists of two sequential components:

1. A branch system, where each branch has a creator, and the creator or another member of the branch can invite new users to the branch.
2. A storage system for each document of its state on different branches with the ability to merge branches.

To describe and test this scenario, the StateGraph, TextAgg, and TextUpd primitives were implemented.

5.2. StateGraph

StateGraph is a primitive described in the context of early scientific works. This primitive relies on the strict ordering of events and trusted temporal branches to track changes in the state of several entities over time, where a state update of one of the entities at time t_1 potentially entails a change in the entire system at times $t_2 > t_1$.

For example, in the target test, StateGraph can be utilized to describe the state of user invitations (where each branch user can invite others) and to describe the state of several branches of a single document.

A visualization of StateGraph for the invitations task is presented in Figure Figure 3.

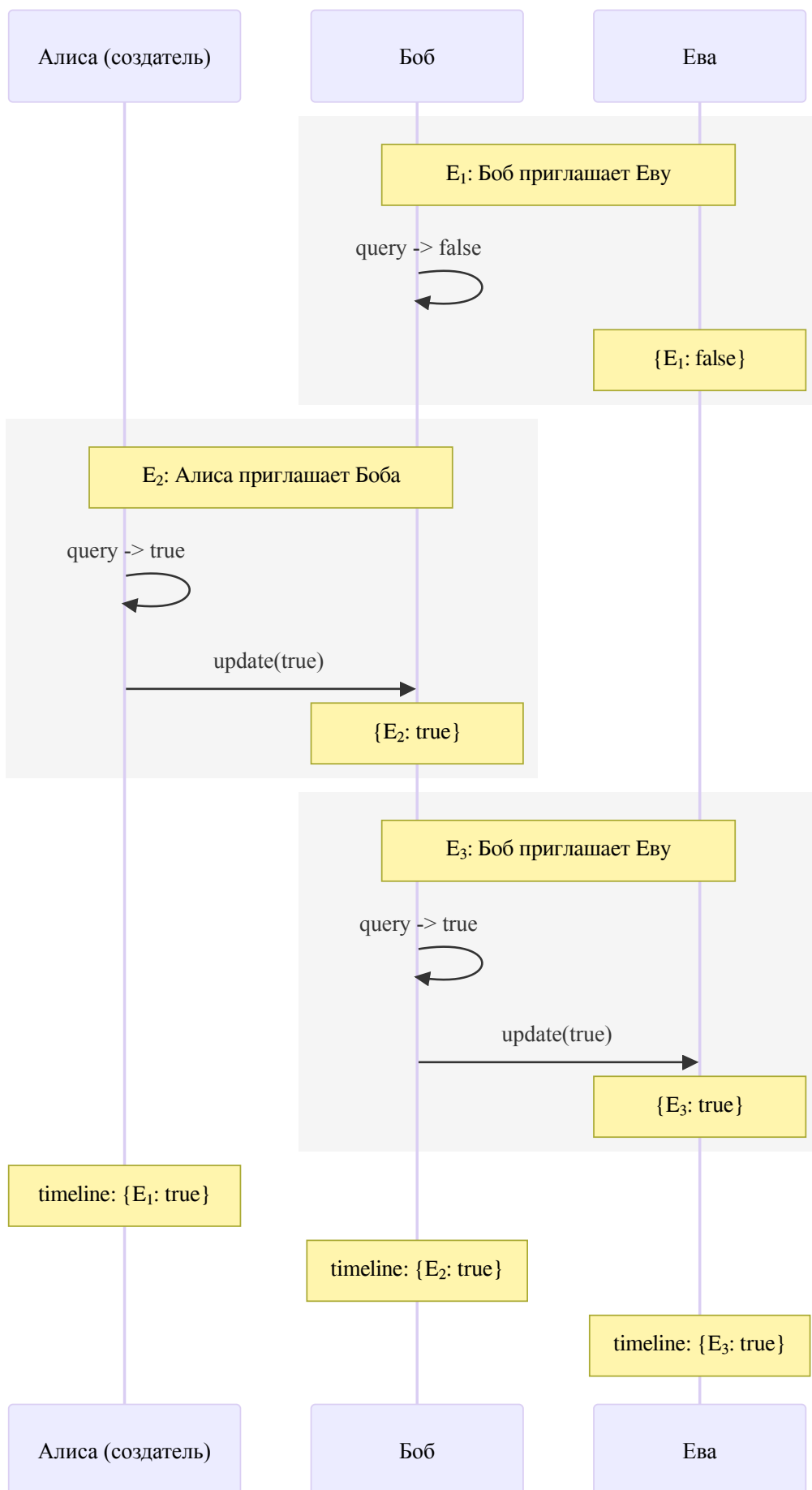


Figure 3: Visualization of StateGraph for the task of inviting users to a branch

The adequacy of the resulting implementation was verified by the inclusion of automated unit tests.

5.3. Collaborative Text Editing

To implement collaborative text editing with a branch merging function, the `AnchorAgg` and `TextAgg` primitives were implemented, used to aggregate `TextUpd` textual edits.

AnchorAgg is a persistent tree data structure aggregating anchor positions at which text insertions were performed by document editors. Each position is identified by a unique `AnchorId` constructed based on the global user event identifier. The tree elements are strictly ordered according to the replicated growing array convention [3], which ensures a deterministic order during concurrent insertions. **TextAgg** associates with each anchor the text stored in it, and also tracks deleted text blocks.

Thus, `AnchorAgg` and `TextAgg` act as two components of a single system, which in combination allow automatic processing of concurrent document updates without conflicts. Such a division of the system into two components is done in order to efficiently implement the operation of merging different branches with minimal conflicts: for each document stored in the system, it is possible to store a single `AnchorAgg`, realizing the variation of the document on different branches exclusively through the different content of `TextAgg`. Furthermore, revoking text content in this system does not lead to critical damage to the document structure.

The already implemented `StateGraph` primitive played a key role, taking over the complexity of maintaining the event history and reverting changes to satisfy the insignificance of cache requirement.

5.4. End-to-End Testing

End-to-end tests are utilized to verify all layers of the system, starting from the compilation of Faden code to sending events into the system and obtaining the materialized Gear result.

5.4.1. Pure Rust

As indicated earlier, the implemented project is agnostic to the Gear execution environment. For initial testing, the counter test was created, utilizing the Rust language itself as the execution environment without using Faden. The test

declares a simple event type and creates a Gear that counts the number of events of this type in the log.

5.4.2. Faden Integration

The `fadeno_counter` test was implemented, which is an evolution of the previous test and verifies the end-to-end integration of Faden as an execution environment. All key events and Gears within this test are described in a Faden source code file. The test reads the definitions from the file, creates corresponding events, and calls the declared Gear, verifying the correct counting of the event list.

5.4.3. Version Control System

Finally, the `wiki` test was implemented, modeling a version control system.

The model includes events of three types:

1. `CreateBranch` — creating a new branch. The branch creator is considered to be the creator of the `CreateBranch` event.
2. `Invite` — inviting a user to a branch. Logically grouped by branches.
3. `Attach` — attaching an edit to a document on a branch or executing a branch merge operation. Logically grouped by documents.

The test includes two Gear definitions:

1. `invited` — a Gear that maps to a specific branch the state of user invitations throughout the existence of the object.
2. `doc_content` — a Gear that maps to a specific document all its versions on all branches throughout the existence of the object.

The key difficulty of this scenario is the logical mismatch of the partitioning models. While `invited` is partitioned by specific branches, the task of tracking the state of a document on a specific branch also requires tracking the state of other branches, since updating a certain branch in the past can affect many other branches due to the support for the merge operation. Such a mismatch of partitionings entails the need for inter-core communication, while at the same time violating the idealized concept of a “static pipeline”, because the `doc_content` Gear of an arbitrary document may require access to the `invited` Gear for arbitrary branches.

Since it is assumed that inviting new users to a branch is a much rarer operation than updating documents, it is reasonable to use asynchronous cache updating. `doc_content` queries the list of invitations from the local core cache, activating

an automatic asynchronous request in the core to update this cache. The asynchronous approach means that the first run of a newly created Gear does not return results due to the lack of a cache, but activates its update, which allows the system to reach a consistent state as early as the second request. In the long term, it is proposed to solve this limitation with a client subscription system to updates.

5.4.4. Testing Results

Based on the results of automated verification using cargo (the Rust build system), the following was confirmed:

1. Successful execution of 44 out of 44 unit tests.
2. Successful execution of 8 out of 8 integration tests.

The full log of test run results is presented in Appendix Section 7.1.

6. Conclusion

As part of this work, the practical implementation of a functional-reactive DBMS architecture in Rust was carried out.

1. **The core functionality of the DBMS cores** Core was implemented, utilizing the strict core isolation paradigm.
2. **The data transmission and routing protocol** was implemented, ensuring exclusive ownership by each core of the processed data. The `WireLocCtx` transmission protocol, used to exchange information over the network or between individual cores, was implemented.
3. **The Faden virtual machine** with integration into the DBMS was implemented.
4. The `StateGraph`, `AnchorAgg`, and `TextAgg` primitives were implemented, their use from Faden was demonstrated, and **end-to-end testing** of the considered architecture was performed.

Future areas of work:

- Implementation of persistent event log storage on disk.
- Demonstration of Faden's capabilities for performing formal verification.
- Implementation of cluster functionality.

Bibliography

- [1] M. Fowler, "Event Sourcing." [Online]. Available: <https://martinfowler.com/eaDev/EventSourcing.html>

- [2] ScyllaDB, “Shard-per-Core Architecture.” [Online]. Available: <https://scylladb.com/product/technology/shard-per-core-architecture/>
- [3] H.-G. Roh, M. Jeon, J.-S. Kim, and J. Lee, “Replicated abstract data types: Building blocks for collaborative applications,” *Journal of Parallel and Distributed Computing*, vol. 71, no. 3, pp. 354–368, 2011, doi: <https://doi.org/10.1016/j.jpdc.2010.12.006>.

7. Appendix

7.1. Log of Automated Testing Run Results

```
Finished `test` profile [unoptimized + debuginfo] target(s) in
2.96s
    Running unittests src/lib.rs (target/debug/deps/
dentrado-9191a00a487cfd8a)

running 44 tests
test fadeno::vm::vm_tests::vm_closure ... ok
test fadeno::vm::vm_tests::vm_if_else ... ok
test fadeno::vm::vm_tests::vm_let_binding ... ok
test fadeno::vm::vm_tests::vm_list ... ok
test fadeno::vm::vm_tests::vm_simple_value ... ok
test fadeno::vm::vm_tests::sg_apply_preserves_stack ... ok
test
utils::state_graph::basic::handler_query_excludes_own_write ... ok
test utils::state_graph::basic::single_event_update ... ok
test
utils::state_graph::basic::conditional_write_changes_on_re_evaluation ...
ok
test utils::state_graph::basic::query_and_propagation ... ok
test
utils::state_graph::basic::no_propagation_when_value_unchanged ...
ok
test
utils::state_graph::basic::bounded_propagation_skips_events_after_next_write .
ok
test utils::state_graph::basic::remove_event_cascades ... ok
test utils::text::tests::char_slice_multibyte ... ok
test utils::text::tests::child_at_offset ... ok
test utils::state_graph::basic::transitive_propagation ... ok
test utils::text::tests::children_ordered_by_offset ... ok
test utils::text::tests::delete_single ... ok
```

```
test utils::state_graph::deps::dep_query_basic ... ok
test utils::text::tests::concurrent_non_overlapping_inserts ... ok
test utils::text::tests::deleted_parent_keeps_children ... ok
test utils::text::tests::diff_delete_all ... ok
test
utils::state_graph::deps::dep_change_detection_and_propagation ...
ok
test utils::state_graph::deps::dep_isolation_between_branches ...
ok
test utils::text::tests::diff_delete_prefix ... ok
test utils::text::tests::diff_delete_suffix ... ok
test utils::text::tests::diff_insert_at_beginning ... ok
test utils::text::tests::diff_insert_at_end ... ok
test utils::state_graph::poc::poc_model_test1 ... ok
test utils::text::tests::diff_insert_in_middle ... ok
test utils::text::tests::diff_insert_into_empty ... ok
test utils::text::tests::diff_no_change ... ok
test utils::text::tests::diff_multi_anchor_preserves_clean ... ok
test utils::text::tests::diff_replace_middle ... ok
test utils::text::tests::diff_utf8 ... ok
test utils::text::tests::empty_doc ... ok
test utils::text::tests::fork_independence ... ok
test utils::text::tests::reapply_is_idempotent ... ok
test utils::text::tests::single_anchor ... ok
test utils::text::tests::two_siblings_ordering ... ok
test utils::text::tests::utf8_content ... ok
test fadeno::vm::vm_tests::compile_id ... ok
test fadeno::vm::vm_tests::loop_fac_6_is_720 ... ok
test utils::state_graph::poc::multishot_converges ... ok

test result: ok. 44 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.09s
```

Running tests/counter.rs (target/debug/deps/counter-
eb56062b62674072)

```
running 2 tests
test malformed_wire_ctx_returns_error_not_panic ... ok
test doc_counter ... ok
```

```
test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.00s
```

Running tests/fadeno_counter.rs (target/debug/deps/

fadeno_counter-266c5ae046cbd05d)

running 2 tests

test wiki_vm ... ok

test wiki_engine ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.58s

Running tests/wiki2.rs (target/debug/deps/wiki2-
f58b4776c252cc7f)

running 4 tests

test doc_content_same_core_e2e ... ok

test invited_simple_e2e ... ok

test doc_content_cross_core_e2e ... ok

test invited_remapping_e2e ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.80s

Running tests/wiki2_advanced.rs (target/debug/deps/
wiki2_advanced-e52658a2a184ecd8)

running 4 tests

test retroactive_invite_point_in_time_same_core_e2e ... ok

test text_agg_merge_cross_core_e2e ... ok

test retroactive_invite_cross_core_e2e ... ok

test multi_user_doc_assembly_cross_core_e2e ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.89s

Doc-tests dentrado

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0
filtered out; finished in 0.00s